Fast Evaluation of Geomagnetic Field Models at Scattered Points

A step-by-step guide to the fused two-dimensional transform

Algorithm description, analysis and benchmarks

reference implementation: `fastfield.py`

10 October 2026

Abstract

Models of the Earth's internal magnetic field, such as the IGRF and CHAOS, are sums of spherical harmonics. Evaluating a model of maximum degree L at M arbitrary points by summing all terms costs about ML^2 operations. For a lithospheric model with $L = 185$ and millions of satellite measurements, this takes minutes to hours. This document explains, step by step, a method that brings the cost per point down to a few hundred to a few thousand operations, *independent of* L, after a set-up of about one second. The method has five steps:

(1) separate the radial dependence into a handful of optimal radial functions;

(2) rewrite the angular parts exactly as two-dimensional Fourier series;

(3) "pre-sharpen" these series and store them on a fine grid;

(4) keep only the half of the grid that is actually needed;

(5) evaluate each point with a small weighted sum over nearby grid values.

The only approximation is the finite size of that sum; its error falls by roughly a factor of ten for each extra grid point used. Steps 3 and 5 together form a non-uniform fast Fourier transform (NUFFT) whose FFT is done in advance, with a window-sum loop built for this problem. For $L = 185$ and 10^6 points, the method is $201\times$ faster than the direct sum at a relative accuracy of 10^{-6}, $98\times$ faster at 10^{-10}, and $65\times$ faster at 3×10^{-14}, which is as accurate as the direct sum itself in double precision. A general NUFFT library can perform the window sums too, but because it cannot exploit the structure of this problem it is about $1.4\times$ slower. For the degree-13 IGRF alone, the direct sum remains competitive. The document assumes only undergraduate calculus, linear algebra and some familiarity with Fourier series.

Contents

1 Introduction **3**

1.1 The problem . . . 3

1.2 Why the direct sum is expensive . . . 3

1.3 The idea in one paragraph . . . 3

1.4 What you get . . . 3

1.5 How to read this document . . . 3

2 Background **4**
2.1 The internal field model 4
2.2 Fourier series, trigonometric polynomials and the FFT 4
2.3 Why the FFT alone is not enough 5

3 Overview of the method **5**

4 Step 1: separating the radial dependence **6**
4.1 The radial factors 6
4.2 Low rank and the SVD 6
4.3 Building the basis in practice 7

5 Step 2: the angular functions as Fourier series **7**
5.1 The doubled sphere 8
5.2 Legendre functions are trigonometric polynomials 8
5.3 Computing the coefficients exactly 9
5.4 Making the functions real 9

6 Step 3: pre-sharpening and the fine grid **9**
6.1 The one-dimensional problem 9
6.2 Why the error is tiny: oversampling 10
6.3 From one dimension to two 11
6.4 Step 3 is a NUFFT 11

7 Step 4: storing only what is needed **12**

8 Step 5: evaluating at the points **13**

9 The complete algorithm **14**

10 Choosing the parameters **15**

11 Cost and memory **16**

12 Accuracy **17**

13 Performance results **18**

14 Practical advice and limitations **23**

15 Summary **23**

A Pole-safe recursions for the Legendre functions **23**

B Chebyshev points and series for the radial functions **24**

C The three-dimensional method (for information) **24**

D Library transforms versus a fused loop **25**

E Why one approximate step is unavoidable **26**

F Using the code **26**

1 Introduction

1.1 The problem

Satellites such as Ørsted, CHAMP and Swarm measure the magnetic field millions of times per year. Geophysicists compare these measurements with *field models*: formulas that predict the field at any position. The standard models, the International Geomagnetic Reference Field (IGRF) [1] and the CHAOS series [2], are written as sums of *spherical harmonics*. To use them we must evaluate the model at every measurement position, which may be millions of times, and inside an inversion this happens again and again.

1.2 Why the direct sum is expensive

A model of maximum degree L has $L(L+2)$ coefficients: 195 for IGRF ($L = 13$), and 34 595 for the static lithospheric part of CHAOS ($L = 185$). Evaluating the model at one point means summing all of these terms, each multiplied by a Legendre function and a sine or cosine. The cost per point therefore grows like L^2, and the total cost for M points like ML^2. For $L = 185$ and $M = 10^6$ points our optimised direct code takes about 86 seconds on two CPU cores; 10^8 points would take several hours.

1.3 The idea in one paragraph

Most of the work in the direct sum is repeated: neighbouring points need almost the same combination of harmonics. The method in this document does that common work once. It turns the model into a set of values on a fine regular grid. After that, evaluating a point only requires a small weighted sum of grid values near the point, for example 11×11 of them, no matter how large L is. The grid values are not ordinary samples of the field. They are "pre-sharpened" so that this short weighted sum reproduces the model to any desired accuracy.

1.4 What you get

For $L = 185$ and 10^6 points (section 13), the speedup over the direct sum is:

- $201\times$ at a relative accuracy of about 6×10^{-7} (about 0.04 nT on a 60 000 nT field);
- $98\times$ at about 2×10^{-11};
- $65\times$ at 3×10^{-14}, the same accuracy as the direct sum computed in double precision.

The set-up takes about one second and is repeated only when the model changes.

1.5 How to read this document

Section 2 reviews the field model and Fourier series. Section 3 gives an overview of the method, and sections 4 to 8 explain the five steps in turn. Each step says *what* is done, *why* it works and *how* it is computed. Section 9 collects everything into two algorithms, and sections 10 to 13 cover parameter choices, cost, accuracy and benchmarks. The appendices contain the Legendre recursions, a description of an earlier three-dimensional version of the method, and some background on why fast methods for scattered points always need one approximate step.

2 Background

2.1 The internal field model

We use spherical coordinates centred on the Earth: radius r, colatitude $\theta \in [0, \pi]$ (angle from the North pole) and longitude φ. Outside the Earth, where there are no electric currents, the internal magnetic field is the negative gradient of a scalar potential, $\mathbf{B} = -\nabla V$, with

$$V(r,\theta,\varphi) = a \sum_{n=1}^{L} \sum_{m=0}^{n} \left(\frac{a}{r}\right)^{n+1} \left[g_n^m \cos m\varphi + h_n^m \sin m\varphi\right] P_n^m(\cos\theta). \tag{1}$$

Here $a = 6371.2\,\mathrm{km}$ is the reference radius, g_n^m and h_n^m are the Gauss coefficients (in nT) supplied by the model, n is the *degree* and m the *order*. The P_n^m are the Schmidt semi-normalised associated Legendre functions. They can be computed by simple recursions (appendix A).

Throughout we write

$$u = \frac{a}{r}, \qquad \gamma_n^m = g_n^m - \mathrm{i}\, h_n^m. \tag{2}$$

Since $g\cos m\varphi + h \sin m\varphi = \mathrm{Re}(\gamma\, e^{\mathrm{i}m\varphi})$, complex numbers let us handle cos and sin together. Taking the gradient of (1) gives the three spherical components of $\mathbf{B}$:

$$B_r = -\frac{\partial V}{\partial r} = \mathrm{Re} \sum_{n,m} u^{n+2}\, \gamma_n^m\, (n+1)\, P_n^m(\cos\theta)\, e^{\mathrm{i}m\varphi}, \tag{3}$$

$$B_\theta = -\frac{1}{r}\frac{\partial V}{\partial \theta} = \mathrm{Re} \sum_{n,m} u^{n+2}\, \gamma_n^m \left(-\frac{\mathrm{d}P_n^m}{\mathrm{d}\theta}\right) e^{\mathrm{i}m\varphi}, \tag{4}$$

$$B_\varphi = -\frac{1}{r\sin\theta}\frac{\partial V}{\partial \varphi} = \mathrm{Re} \sum_{n,m} u^{n+2}\, \gamma_n^m \left(-\mathrm{i}m\, \frac{P_n^m(\cos\theta)}{\sin\theta}\right) e^{\mathrm{i}m\varphi}. \tag{5}$$

(The North, East and downward components are $B_N = -B_\theta$, $B_E = B_\varphi$ and $B_C = -B_r$.) All three have the same shape, which we will use repeatedly:

$$\boxed{B_c = \mathrm{Re} \sum_{n=1}^{L} \sum_{m=0}^{n} \underbrace{u^{n+2}}_{\text{radial}}\, \gamma_n^m\, \underbrace{\Theta_c^{nm}(\theta)}_{\text{colatitude}}\, \underbrace{e^{\mathrm{i}m\varphi}}_{\text{longitude}}, \qquad c \in \{r,\theta,\varphi\},} \tag{6}$$

with $\Theta_r^{nm} = (n+1)P_n^m$, $\Theta_\theta^{nm} = -\mathrm{d}P_n^m/\mathrm{d}\theta$ and $\Theta_\varphi^{nm} = -\mathrm{i}m\, P_n^m / \sin\theta$.

Remark 1 (The direct sum)**.** Evaluating (6) at one point means computing all P_n^m, their derivatives, $\cos m\varphi$, $\sin m\varphi$ and u^n by recursion, then summing about $L^2/2$ terms. This is what our "direct" reference code does, as efficiently as possible: compiled, multithreaded, with pole-safe recursions. It agrees with the official IGRF software to $10^{-10}\,\mathrm{nT}$.

2.2 Fourier series, trigonometric polynomials and the FFT

A *trigonometric polynomial* of degree N is a function

$$f(x) = \sum_{k=-N}^{N} c_k\, e^{\mathrm{i}kx}, \tag{7}$$

which is 2π-periodic. Two facts about them are used throughout.

Lemma 1 (Exact sampling)**.** *If f has degree N and $n \geq 2N+1$, the coefficients are recovered exactly from n equally spaced samples $x_j = 2\pi j/n$:*

$$c_k = \frac{1}{n} \sum_{j=0}^{n-1} f(x_j)\, e^{-\mathrm{i}kx_j}, \qquad |k| \leq N. \tag{8}$$

Proof. Insert (7) and use $\frac{1}{n}\sum_j e^{\mathrm{i}(k'-k)x_j} = 1$ if $k'-k$ is a multiple of n and 0 otherwise. Since $|k'-k| \le 2N < n$, only $k' = k$ survives. □

If n were too small, a frequency k' would be confused with $k' \pm n$. This is *aliasing*: on a grid of n points, the frequencies k and $k+n$ look identical.

The sums in lemma 1, and the reverse sums that produce the samples from the coefficients, are discrete Fourier transforms. The *fast Fourier transform* (FFT) [3] computes them in $\mathcal{O}(n \log n)$ operations instead of $\mathcal{O}(n^2)$, in one and in several dimensions. It works because the grid points are equally spaced.

2.3 Why the FFT alone is not enough

If our points lay on a regular latitude–longitude grid, a fast spherical harmonic transform would evaluate the model at all of them exactly and quickly [4, 5]. Satellite data, however, are *scattered*: every point has its own r, θ and φ. The FFT cannot be applied to them directly. The method below therefore uses the FFT to prepare a grid, then moves from the grid to the scattered points with a small, accurate local sum.

3 Overview of the method

Figure 1 shows the structure. The work splits into two phases.

- The **set-up** depends only on the model and on the radial range of the data. It is done once and takes about a second even for $L = 185$.

  Step 1: Separate the radial dependence into ρ optimal radial functions $b_i(u)$ (section 4).

  Step 2: Write the angular part that goes with each b_i as an exact 2-D Fourier series (section 5).

  Step 3: Pre-sharpen each Fourier series and transform it by FFT to a fine grid (section 6).

  Step 4: Keep only the rows of the grid that points can actually reach (section 7).

- The **evaluation** is done for each point.

  Step 5: Compute the ρ radial values $b_i(a/r)$, form a $w \times w$ weighted sum of nearby grid values for all channels at once, and combine the two (section 8).

Key idea. After the set-up, the model has become a table of numbers on a regular grid. Each evaluation reads a small, fixed-size patch of that table. Its cost does not depend on L, which is where the speed comes from.

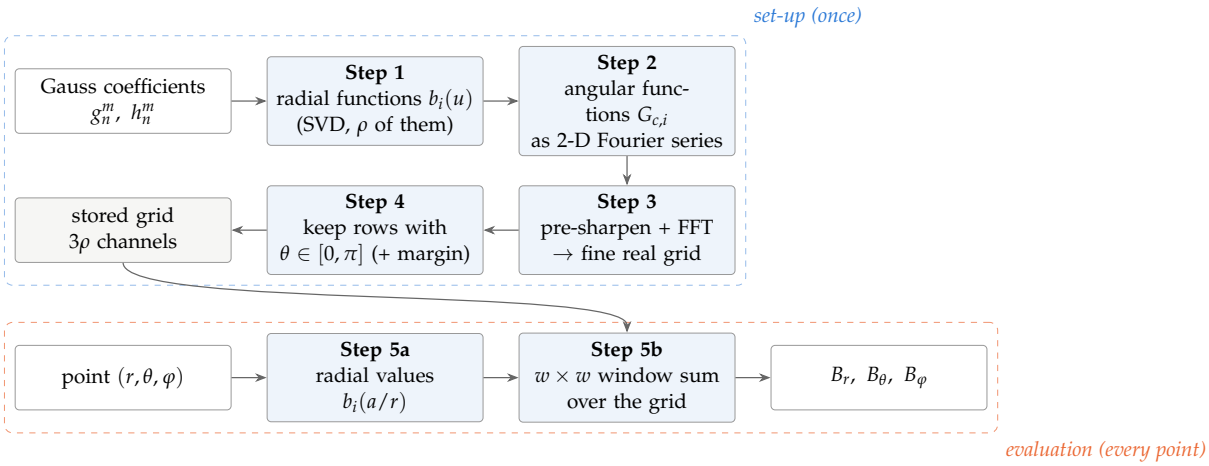


Figure 1: Structure of the method. Everything in the upper box depends only on the model and is done once. Each point then needs only Step 5.

4 Step 1: separating the radial dependence

4.1 The radial factors

In (6), the radius enters only through the powers u^{n+2}, with a different power for every degree n. A satellite data set covers a shell $r_{\min} \le r \le r_{\max}$, for example 250–800 km altitude. So $u = a/r$ lies in an interval $[u_0, u_1]$ with $u_0 = a/r_{\max}$ and $u_1 = a/r_{\min}$. For the 250–800 km shell this is $[0.888, 0.962]$, a narrow interval.

On a narrow interval, the L functions $u^3, u^4, \ldots, u^{L+2}$ look very much alike: they are all smooth, increasing curves. Intuitively, they can be built from a few "shape" functions. That is exactly what the singular value decomposition (SVD) finds.

4.2 Low rank and the SVD

Suppose we can find ρ functions $b_1(u), \ldots, b_\rho(u)$ and numbers $\beta_{i,n}$ with

$$u^{n+2} \approx \sum_{i=1}^{\rho} \beta_{i,n}\, b_i(u) \qquad \text{for all } u \in [u_0, u_1] \text{ and all } n = 1, \ldots, L. \tag{9}$$

Substituting into (6) and swapping the order of the sums gives

$$\boxed{B_c(r,\theta,\varphi) = \sum_{i=1}^{\rho} b_i(u)\; G_{c,i}(\theta,\varphi), \qquad G_{c,i}(\theta,\varphi) = \operatorname{Re} \sum_{n,m} \beta_{i,n}\, \gamma_n^m\, \Theta_c^{nm}(\theta)\, e^{\mathrm{i}m\varphi}.} \tag{10}$$

The radius now appears only in the ρ numbers $b_i(u)$. The angular functions $G_{c,i}$ do not depend on r at all. If ρ is small, we have turned a 3-D problem into ρ two-dimensional ones.

To find the best b_i, sample the functions at $K+1$ points $u_0 \le u_j \le u_1$ and put them in a matrix

$$A_{jn} = \sqrt{c_j}\, w_n\, u_j^{n+2} \qquad (j = 0, \ldots, K;\ n = 1, \ldots, L). \tag{11}$$

- Each column of A is one radial function.
- The weights $w_n = (n+1)\big(\sum_m |\gamma_n^m|^2\big)^{1/2}$ measure how strongly degree n contributes to the field, so the approximation is most accurate where it matters.
- The factors $\sqrt{c_j}$ are quadrature weights explained in appendix B.

The SVD writes

$$A = U\,S\,W^{\mathsf{T}} = \sum_i s_i\,\mathbf{u}_i\,\mathbf{w}_i^{\mathsf{T}}, \qquad s_1 \geq s_2 \geq \cdots \geq 0, \tag{12}$$

where the columns $\mathbf{u}_i$ of U and $\mathbf{w}_i$ of W are orthonormal. Keeping only the first ρ terms gives the best approximation of A by a matrix of rank ρ (the Eckart–Young theorem [6]). The error is governed by the first omitted singular value $s_{\rho+1}$. We choose ρ as the number of singular values with $s_i > \tau\, s_1$, where τ is a small multiple of the target accuracy. Reading off the columns gives $b_i(u_j) = U_{ji}/\sqrt{c_j}$ and $\beta_{i,n} = s_i W_{ni}/w_n$.

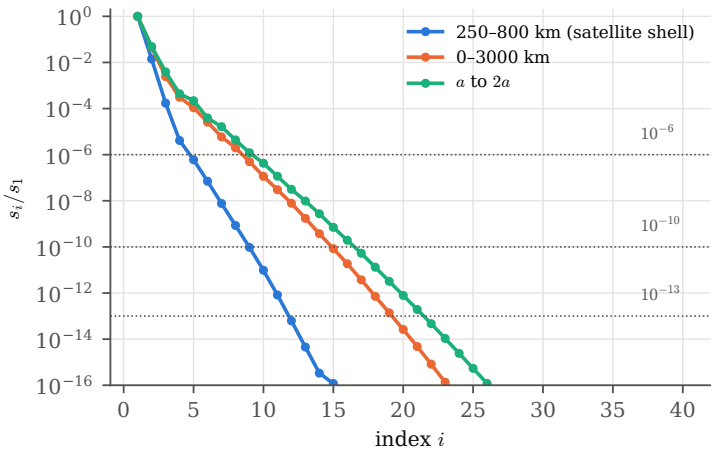


Figure 2: Singular values s_i/s_1 of the weighted radial functions for $L = 185$. For the satellite shell (blue) they fall below 10^{-6} after 5 terms and below 10^{-13} after about 13. Thicker shells need more terms (orange, green).

Figure 2 shows why this works so well. For the satellite shell the singular values fall extremely fast: about 5 radial functions suffice for an accuracy of 10^{-6} and about 12 for 10^{-13}, although there are 185 different powers u^{n+2}. For thicker shells more terms are needed, because the curves u^{n+2} differ more over a wider interval.

4.3 Building the basis in practice

- **Sample points.** We use Chebyshev points: $u_j = u_c + u_d \cos(\pi j/K)$, with $u_c = \frac{1}{2}(u_0 + u_1)$ and $u_d = \frac{1}{2}(u_1 - u_0)$. On these points, smooth functions can be represented by Chebyshev series that converge very fast (appendix B). The number of points $K + 1$ is increased until the functions are resolved to the target accuracy; typically $K = 6$–20.
- **Evaluating b_i at an arbitrary radius.** Each basis function is stored as a short Chebyshev series in $t = (u - u_c)/u_d \in [-1, 1]$, $b_i(u) = \sum_{k=0}^{K} C_{ik} T_k(t)$. The Chebyshev polynomials satisfy $T_0 = 1$, $T_1 = t$ and $T_{k+1} = 2tT_k - T_{k-1}$, so computing all ρ values at a point costs about ρK operations.

5 Step 2: the angular functions as Fourier series

Each angular function in (10) is a sum of products of a function of θ and $e^{im\varphi}$. The φ part is already a Fourier series. This section shows that the θ part is one too, with one extra trick.

5.1 The doubled sphere

Colatitude only runs from 0 to π, so functions of θ are not naturally periodic. The trick is to let θ run over a full circle, $[0, 2\pi)$, and to evaluate $\cos\theta$ and $\sin\theta$ there as usual. Then $\sin\theta$ is negative on $(\pi, 2\pi)$. This is called the *doubled sphere*: the second half is a mirror image of the first, as fig. 3 shows. The same device underlies the fast spherical Fourier transforms of Kunis and Potts [11]. Points of the Earth only ever have $\theta \in [0, \pi]$, but the extended functions are periodic, and that is what Fourier series need.

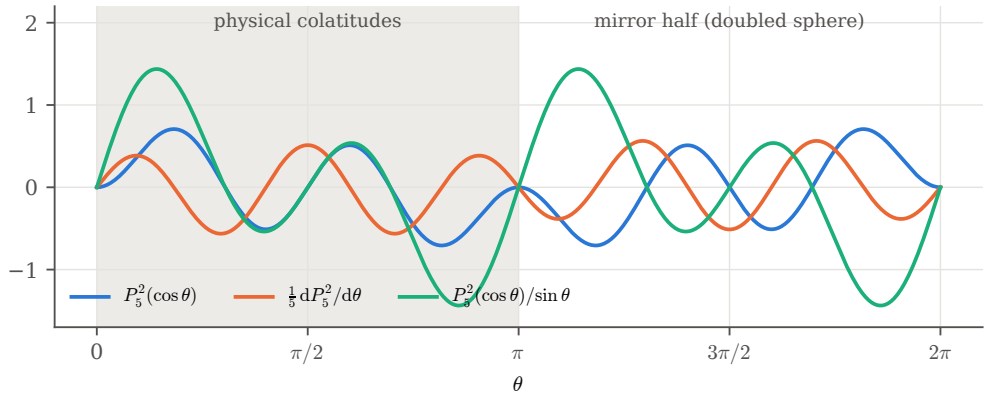


Figure 3: Legendre functions continued to the doubled sphere $\theta \in [0, 2\pi)$, here for $n = 5$, $m = 2$. All three functions are smooth and 2π-periodic, including at the poles $\theta = 0$ and $\theta = \pi$. The quotient $P/\sin\theta$ is regular even though $\sin\theta = 0$ at the poles.

5.2 Legendre functions are trigonometric polynomials

Lemma 2. *On the doubled sphere, $P_n^m(\cos\theta)$ and $\mathrm{d}P_n^m/\mathrm{d}\theta$ are trigonometric polynomials of degree at most n. For $m \geq 1$, $P_n^m(\cos\theta)/\sin\theta$ is a trigonometric polynomial of degree at most $n - 1$.*

Proof. From its definition, $P_n^m(\cos\theta) = c\,\sin^m\theta\, q(\cos\theta)$, where q is an ordinary polynomial of degree $n - m$ and c a constant. Each product $\sin^m\theta\cos^j\theta$ with $j \leq n - m$ can be expanded, using $\sin\theta = (e^{\mathrm{i}\theta} - e^{-\mathrm{i}\theta})/(2\mathrm{i})$ and $\cos\theta = (e^{\mathrm{i}\theta} + e^{-\mathrm{i}\theta})/2$, into powers $e^{\mathrm{i}p\theta}$ with $|p| \leq m + j \leq n$. Differentiating $e^{\mathrm{i}p\theta}$ gives $\mathrm{i}p\,e^{\mathrm{i}p\theta}$, so the derivative has the same degree. Finally, for $m \geq 1$, $P_n^m/\sin\theta = c\,\sin^{m-1}\theta\, q(\cos\theta)$ has degree at most $n - 1$. □

Two consequences matter.

- **No pole problem.** The factor $1/\sin\theta$ in B_φ is cancelled exactly. Our recursions compute $P_n^m/\sin\theta$ directly, without ever dividing by $\sin\theta$ (appendix A).
- **Exact Fourier series.** Every angular function is an exact 2-D trigonometric polynomial of degree at most L in both θ and φ:

$$G_{c,i}(\theta,\varphi) = \mathrm{Re}\sum_{p=-L}^{L}\sum_{m=0}^{L} H_{c,i}[p,m]\,e^{\mathrm{i}(p\theta+m\varphi)}, \qquad H_{c,i}[p,m] = \sum_{n=\max(m,1)}^{L} \beta_{i,n}\,\gamma_n^m\,\widehat{\Theta}_c^{nm}[p], \tag{13}$$

where $\widehat{\Theta}_c^{nm}[p]$ are the Fourier coefficients of $\Theta_c^{nm}(\theta)$.

5.3 Computing the coefficients exactly

By lemma 1, the Fourier coefficients of a degree-L function are obtained *exactly* from $2L+2$ equally spaced samples. We sample at $\theta_j = (j+\frac{1}{2})\,2\pi/(2L+2)$; the half-step shift keeps the samples off the poles, which is harmless and slightly tidier. For each order m:

1. compute P_n^m, $\mathrm{d}P_n^m/\mathrm{d}\theta$ and $P_n^m/\sin\theta$ at all θ_j for $n = m, \dots, L$, using the recursions;
2. take one FFT of each row to get $\widehat{\Theta}_c^{nm}[p]$ for $|p| \le L$;
3. form $H_{c,i}[\,\cdot\,, m]$ by the sum over n in (13), which is a small matrix product.

No approximation is made in this step.

5.4 Making the functions real

The functions $G_{c,i}$ are real, but (13) writes them as the real part of a complex series that uses only $m \ge 0$. For the next step it is simpler and faster to write them as an ordinary two-sided series with real values:

$$G_{c,i}(\theta,\varphi) = \sum_{p=-L}^{L} \sum_{m=-L}^{L} F_{c,i}[p,m]\, e^{\mathrm{i}(p\theta + m\varphi)}. \tag{14}$$

Using $\operatorname{Re} z = \frac{1}{2}(z+\bar{z})$, the new coefficients are

$$F[p,m] = \tfrac{1}{2}H[p,m], \qquad F[p,-m] = \tfrac{1}{2}\overline{H[-p,m]} \qquad (m>0), \tag{15}$$

$$F[p,0] = \tfrac{1}{2}\big(H[p,0] + \overline{H[-p,0]}\big). \tag{16}$$

They satisfy $F[-p,-m] = \overline{F[p,m]}$, which is exactly the condition for the sum to be real. The gain shows up in Step 3: the grid then holds real numbers instead of complex ones, and evaluation needs no extra phase factor.

6 Step 3: pre-sharpening and the fine grid

We now face the central question: given a Fourier series, how do we evaluate it quickly at many *arbitrary* points? We explain the idea in one dimension first, then extend it to two.

6.1 The one-dimensional problem

Let $f(x) = \sum_{|k|\le N} c_k e^{\mathrm{i}kx}$. Evaluating it directly at one point costs $2N+1$ operations. We want something cheaper when there are many points.

First attempt: sample and interpolate. Use the FFT to compute f on a fine grid $x_l = lh$ with $h = 2\pi/n_f$. Then estimate $f(x)$ from nearby grid values, as one would interpolate in a table. With a few neighbours, ordinary (e.g. polynomial) interpolation is not very accurate. The *exact* formula for reconstructing a trigonometric polynomial from its samples does exist, but its kernel, the Dirichlet kernel, decays slowly, so every grid value contributes. That is no cheaper than the direct sum.

The trick: change what we store. Choose a smooth *window* function $\phi(x)$ that is zero outside a short interval of width wh, i.e. w grid steps. Store on the grid, not the samples $f(x_l)$, but the *pre-sharpened* values

$$b_l = \sum_{k=-N}^{N} c_k \, \frac{h}{\hat{\phi}(k)} \, e^{\mathrm{i}kx_l}, \qquad \hat{\phi}(k) = \int_{-\infty}^{\infty} \phi(x)\, e^{-\mathrm{i}kx}\, \mathrm{d}x, \tag{17}$$

where $\hat{\phi}$ is the Fourier transform of the window. This is again an FFT, of the modified coefficients $c_k h/\hat{\phi}(k)$. Then evaluate

$$\tilde{f}(x) = \sum_l b_l \, \phi(x - x_l), \tag{18}$$

which involves only the w grid points within the window around x.

Why it works. The Poisson summation formula states that summing a function over a regular grid equals summing its Fourier transform over the "reciprocal" grid: $\sum_l g(lh) = \frac{1}{h}\sum_j \hat{g}(2\pi j/h)$. Applying it to $g(y) = \phi(x-y)e^{\mathrm{i}ky}$ gives the identity

$$\sum_l \phi(x - x_l)\, e^{\mathrm{i}kx_l} = \frac{1}{h} \sum_{j=-\infty}^{\infty} \hat{\phi}(k - jn_f)\, e^{\mathrm{i}(k-jn_f)x}. \tag{19}$$

Inserting (17) into (18) and using (19) gives

$$\tilde{f}(x) = f(x) + \underbrace{\sum_k c_k \sum_{j\neq 0} \frac{\hat{\phi}(k-jn_f)}{\hat{\phi}(k)}\, e^{\mathrm{i}(k-jn_f)x}}_{\text{error}}. \tag{20}$$

The $j = 0$ term reproduces f exactly, because the factor $h/\hat{\phi}(k)$ in b_l cancels the factor $\hat{\phi}(k)/h$ produced by the window sum. That is the purpose of pre-sharpening. The remaining terms are copies of the spectrum shifted by multiples of n_f: the aliases.

Pre-sharpening in one sentence. Dividing each Fourier coefficient by $\hat{\phi}(k)$ before going to the grid undoes, in advance, the blurring that the short window sum will cause. What is left is an error made only of aliases, and these can be made as small as we like.

6.2 Why the error is tiny: oversampling

The error in (20) is small if $\hat{\phi}$ is large on the band $|k| \le N$ and tiny at the alias frequencies $|k - jn_f| \ge n_f - N$.

- **Oversampling.** We make the grid finer than lemma 1 requires, $n_f = \sigma(2N+1)$ with an *oversampling factor* $\sigma = 2$. This leaves a gap between the band and the nearest alias (fig. 4, right).
- **A window with a sharp spectrum.** A window that is short in space cannot have a perfectly sharp spectrum (an uncertainty principle), but some windows come close. We use the "exponential of semicircle" window [10]:

$$\phi(x) = \exp\Big(\beta\big(\sqrt{1-z^2}-1\big)\Big), \qquad z = \frac{x}{wh/2}, \quad |z| \le 1, \tag{21}$$

  and $\phi = 0$ for $|z| > 1$. Its spectrum is nearly flat and then drops by a large factor at frequency $\approx \beta n_f/(\pi w)$.

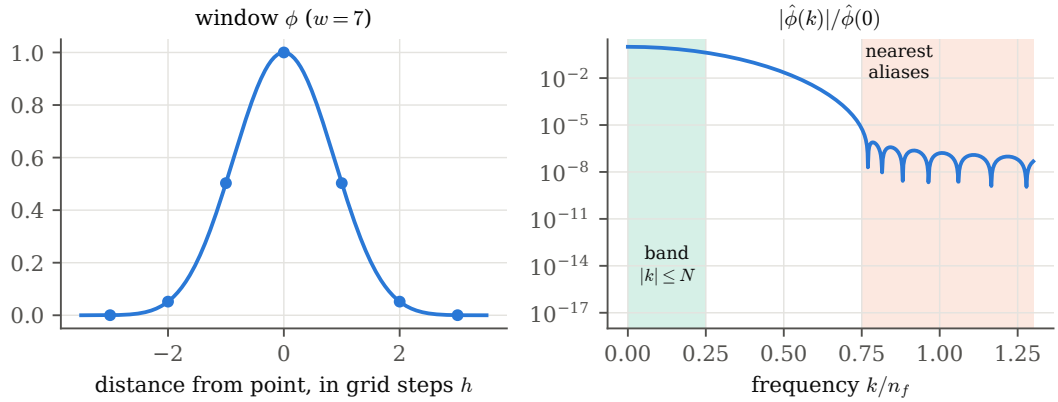


Figure 4: *Left:* the window (21) for $w = 7$, with the 7 grid points it touches (dots). *Right:* its spectrum. It is nearly constant on the band of the function (green), and seven orders of magnitude smaller at the aliases (orange). That ratio is the error of the method.

- **Placing the drop.** We choose $\beta = \pi(1 - \frac{1}{2\sigma})\, w$, so that the drop sits exactly at the nearest alias $n_f - N$. The ratio $\hat{\phi}(k - jn_f)/\hat{\phi}(k)$ is then of order $e^{-\beta}$, i.e. roughly 10^{-w} for $\sigma = 2$.

Each extra grid point in the window buys about one more correct digit, as the measurements in section 12 confirm.

Example 1 (One dimension)**.** Figure 5 takes a random real trigonometric polynomial of degree $N = 10$, on a grid of $n_f = 42$ points ($\sigma = 2$). The pre-sharpened values b_l (squares) differ visibly from plain samples (circles); they are not samples of f. Yet the 7-point window sum (18) reproduces f everywhere with an error of about 10^{-6}. With 10 points the error is 10^{-9}; with 4 points it is 10^{-3}.

6.3 From one dimension to two

In two dimensions we use the product window $\phi(\theta)\,\phi(\varphi)$, whose Fourier transform is also a product. Everything above carries over:

1. multiply each coefficient $F_{c,i}[p, m]$ of (14) by $\dfrac{h_1}{\hat{\phi}_1(p)} \cdot \dfrac{h_2}{\hat{\phi}_2(m)}$ (pre-sharpening);
2. place the result in an $n_{f,1} \times n_{f,2}$ array of frequencies, with $n_{f,d} \approx \sigma(2L + 1)$, and apply a 2-D inverse FFT. Because the coefficients have the Hermitian symmetry of section 5.4, the result is a real grid $g_{c,i}[l_1, l_2]$;
3. a point (θ, φ) is then evaluated as $\sum_{l_1,l_2} g_{c,i}[l_1, l_2]\, \phi(\theta - l_1 h_1)\, \phi(\varphi - l_2 h_2)$, which involves only $w \times w$ grid values.

The set-up performs one such FFT for each of the 3ρ "channels" (c, i): three field components times ρ radial functions. This is the only place where FFTs are used, and they are done once.

6.4 Step 3 is a NUFFT

Pre-sharpening, FFT and window sum are exactly the three stages of a *type-2 non-uniform FFT* (NUFFT) [8, 9, 10], the standard tool for evaluating Fourier series at scattered points. The only difference from a textbook NUFFT is the order of work. A NUFFT library normally receives new Fourier coefficients on every call, so it repeats the pre-sharpening and FFT each time.

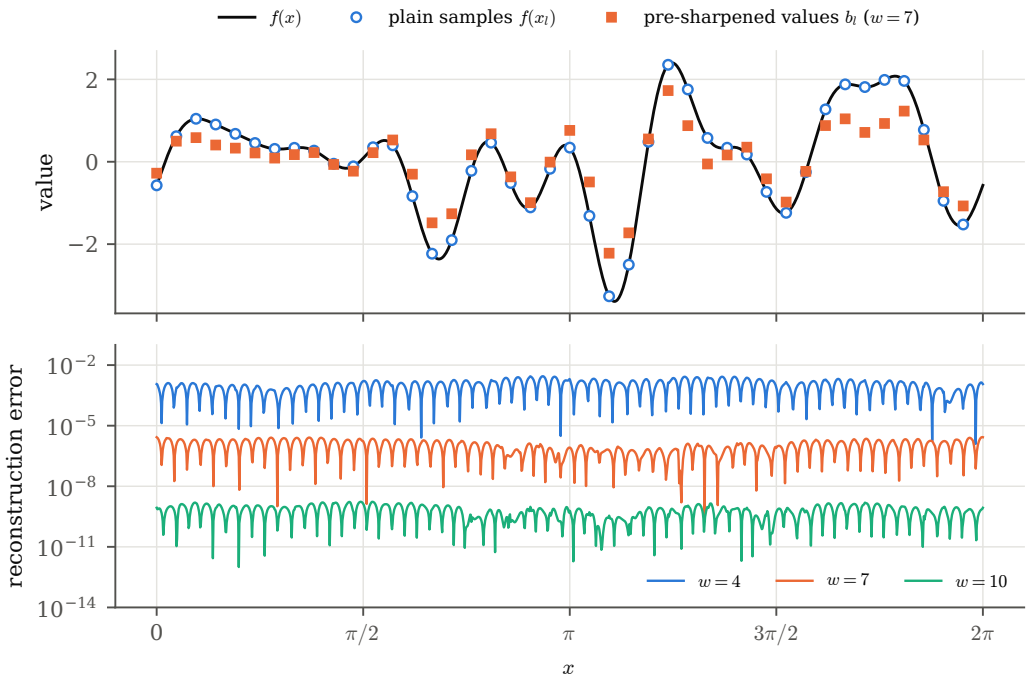


Figure 5: Pre-sharpening in one dimension. *Top:* a trigonometric polynomial, its plain samples and the pre-sharpened grid values for $w = 7$. *Bottom:* error of the window reconstruction (18) at 1500 points, for three window widths.

Our coefficients are fixed by the model, so these two stages are done once, in the set-up, and only the window sum is repeated for each batch of points.

Why a purpose-built window sum? General NUFFT libraries such as FINUFFT [10] are designed for "one grid, many points". Our problem is "3ρ grids, the same points, combined at the end". A library can be used, and we provide this as an option (section 8), but it treats each grid as a separate transform: it recomputes every point's window weights for each grid, and returns per-grid results that must be read back for the radial sum. Step 5 instead uses a loop written for this structure: the window weights are computed once per point, and all grids are summed in one pass together with the radial combination. Measured, this is about 1.4× faster (table 3).

Using FINUFFT's window instead. When the library is used, the pre-sharpening factors must match *its* window exactly. We obtain them by asking the library itself: we place a pure Fourier mode e^{ikx} on the grid and let FINUFFT interpolate it at a few points. The result is $r_k\, e^{ikx}$, apart from aliases of size $\approx \varepsilon$, where r_k is precisely the factor by which the window sum scales frequency k. Dividing every Fourier coefficient by r_k (instead of by $\hat{\phi}(k)/h$) makes the library's window sum reproduce the series, without any knowledge of its internal formulas.

7 Step 4: storing only what is needed

The grid covers the whole doubled sphere, $\theta \in [0, 2\pi)$, but real points have $\theta \in [0, \pi]$. A point's window reaches at most $w/2$ grid steps on either side. So the only rows that can ever be read are those with

$$-\tfrac{w}{2}h_1 \lesssim \theta_{l_1} \lesssim \pi + \tfrac{w}{2}h_1. \tag{22}$$

We store just these rows, about half of the grid plus a margin of w rows (fig. 6). The margin rows below $\theta = 0$ and above $\theta = \pi$ hold values from the mirror half of the doubled sphere. They are needed by points close to a pole, whose window straddles the pole. Because the angular functions are smooth trigonometric polynomials across the poles (lemma 2), the window sum is just as accurate there as anywhere else.

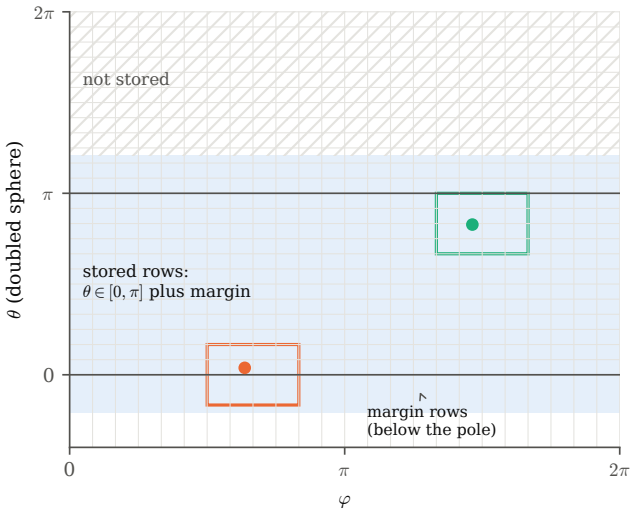


Figure 6: The grid on the doubled sphere. Only the shaded rows are stored. Boxes show the $w \times w$ windows of two points. The window of the point near the North pole reaches into the margin rows below $\theta = 0$.

This halves the memory. The real-valued grid of section 5.4 has the same size as the earlier complex one (twice as many columns, half the bytes per value), so the net effect is a factor of two: for $L = 185$, $w = 14$ and $\rho = 14$ the stored grid shrinks from 190 MB to 98 MB (table 5).

8 Step 5: evaluating at the points

For a point (r, θ, φ) the evaluation does the following.

1. **Window weights.** Find the first grid row and column of the window, $\ell_1 = \lceil \theta/h_1 - w/2 \rceil$ and $\ell_2 = \lceil \varphi/h_2 - w/2 \rceil$, and compute the w weights $\phi(\theta - (\ell_1 + a)h_1)$ and the w weights $\phi(\varphi - (\ell_2 + b)h_2)$, for $a, b = 0, \ldots, w-1$. This takes $2w$ exponentials.

2. **Window sum, all channels at once.** The grid is stored so that the 3ρ channel values of one grid node are next to each other in memory. One pass over the $w \times w$ nodes accumulates

$$s_{c,i} = \sum_{a,b=0}^{w-1} \phi_1[a]\, \phi_2[b]\, g_{c,i}[\ell_1 + a,\ \ell_2 + b] \tag{23}$$

   for all channels together (column indices are taken modulo $n_{f,2}$, since φ is periodic). This is the main cost: $3\rho\, w^2$ multiply–adds.

3. **Radial values.** With $t = (a/r - u_c)/u_d$, compute $T_0(t), \ldots, T_K(t)$ by recurrence and $b_i = \sum_k C_{ik} T_k(t)$.

4. **Combine.** $B_c = \sum_{i=1}^{\rho} b_i\, s_{c,i}$ for $c = r, \theta, \varphi$.

Two implementations of the window sum. The code offers two ways to carry out item 2.

- **Own fused loop (default).** A compiled loop that, for each point, computes the window weights once and sweeps all 3ρ channels together, followed directly by the radial sum. Points are first sorted into small blocks of the grid (a counting sort, $\mathcal{O}(M)$) so that consecutive points read nearby memory, and blocks of points are processed in parallel on all CPU cores. Because the per-point work is done once and no intermediate results leave the processor's caches, it is about $1.4\times$ faster than the library (table 3).
- **FINUFFT (optional).** The stored grid is passed to FINUFFT's interpolation-only mode as a batch of transforms that share the same points. Since the window weights are real, two real channels are packed into one complex grid, $g_{\text{pair}} = g_{c,i} + \mathrm{i}\, g_{c',i'}$: the real part of the result belongs to the first channel and the imaginary part to the second. This halves the number of transforms to $\lceil 3\rho/2 \rceil$. The library sorts the points for good memory access, evaluates its window with vectorised code and uses all CPU cores. The stored band of rows is passed as a grid of its own, with the colatitude coordinate shifted and rescaled so that one library grid step equals one row, and with two extra margin rows on each side.

9 The complete algorithm

Algorithm 1 Set-up (once per model and radial range)

Require: Gauss coefficients g_n^m, h_n^m ($n \le L$); reference radius a; radial range $[r_{\min}, r_{\max}]$; target accuracy ε

$u_0 \leftarrow a/r_{\max}$, $u_1 \leftarrow a/r_{\min}$; $\gamma_n^m \leftarrow g_n^m - \mathrm{i}h_n^m$; $w_n \leftarrow (n+1)\|\gamma_n\|$

Step 1: sample $w_n u^{n+2}$ at $K+1$ Chebyshev points; SVD; keep ρ terms; store C_{ik} and $\beta_{i,n}$

Step 2: $\theta_j \leftarrow (j+\frac{1}{2})\frac{2\pi}{2L+2}$, $j = 0..2L+1$

for $m = 0, \dots, L$ **do**

  P_n^m, $\mathrm{d}P_n^m/\mathrm{d}\theta$, $P_n^m/\sin\theta$ at all θ_j, $n = m..L$ ▷ recursions, appendix A

  FFT along $j \to \widehat{\Theta}_c^{nm}[p]$, $|p| \le L$

  $H_{c,i}[p,m] \leftarrow \sum_n \beta_{i,n}\gamma_n^m \widehat{\Theta}_c^{nm}[p]$ ▷ three small matrix products

choose w from ε (table 1); $\sigma \leftarrow 2$; $\beta \leftarrow \pi(1-\frac{1}{2\sigma})w$; $n_{f,d} \gtrsim \sigma(2L+1)$

for each channel (c,i) **do**

  $F \leftarrow$ real two-sided coefficients of $H_{c,i}$ ▷ section 5.4

  Step 3: $F[p,m] \leftarrow F[p,m] \cdot h_1 h_2/(\hat{\phi}_1(p)\hat{\phi}_2(m))$; zero-pad; real 2-D inverse FFT $\to$ grid

  Step 4: store rows $\lceil -w/2 \rceil \dots \lceil \pi/h_1 - w/2 \rceil + w - 1$ into $g[\,\cdot,\cdot,(c,i)]$

The method as a product of matrices. Collecting the coefficients into a vector $\boldsymbol{\gamma}$, the whole method computes

$$\mathbf{B} = S\, \mathcal{F}^{-1}\, D\, \Lambda\, R\, \boldsymbol{\gamma}, \tag{24}$$

where the factors are:

- R: the radial split (Step 1);
- Λ: the Legendre-to-Fourier map (Step 2), which is exact;
- D: the diagonal pre-sharpening;
- $\mathcal{F}^{-1}$: the inverse FFT (Step 3);
- S: a sparse matrix holding the window weights and the radial values (Step 5), with $3\rho w^2$ non-zeros per point.

Algorithm 2 Evaluation at M points (Step 5)

Require: points $(r_j, \theta_j, \varphi_j)$ with $r_j \in [r_{\min}, r_{\max}]$

sort the points by grid block
for all points j, in parallel **do**
  $\ell_1 \leftarrow \lceil \theta_j/h_1 - w/2 \rceil$, $\ell_2 \leftarrow \lceil \varphi_j/h_2 - w/2 \rceil$; window weights $\phi_1[0..w-1]$, $\phi_2[0..w-1]$
  $\mathbf{s} \leftarrow \mathbf{0} \in \mathbb{R}^{3\rho}$
  for $a = 0..w-1$ **do**
    for $b = 0..w-1$ **do**
      $\mathbf{s} \leftarrow \mathbf{s} + \phi_1[a]\,\phi_2[b]\; g[\ell_1 + a,\ (\ell_2 + b) \bmod n_{f,2},\ :]$ ▷ all 3ρ channels
  $t \leftarrow (a/r_j - u_c)/u_d$; $b_i \leftarrow \sum_k C_{ik} T_k(t)$
  $B_{c,j} \leftarrow \sum_i b_i\, s_{c,i}$ for $c = r, \theta, \varphi$

Only S and the truncation of R are approximations. Everything to the right of S is applied once in the set-up, so evaluation is a single sparse matrix–vector product. The transpose of (24), needed when fitting a model to data by least squares, has the same structure.

10 Choosing the parameters

The user chooses the target relative accuracy ε and the radial range. Everything else follows:

- **Window width w:** the smallest w whose *measured* error is below ε (table 1): $w = 7$ for $\varepsilon = 10^{-6}$, 11 for 10^{-10} and 14 for 10^{-13}. (With the optional FINUFFT backend, ε is passed to the library, which chooses the same widths.) The measured error is practically the same for $L = 13$ and $L = 185$, so one table serves all models.
- **Oversampling $\sigma = 2$.** A larger σ allows a slightly smaller w for the same accuracy, but makes the stored grid larger in proportion to σ^2. With $\sigma = 2$, machine precision is already reached at $w = 14$.
- **Radial rank ρ:** from the SVD, keeping singular values above 0.1ε (fig. 2).
- **Radial range:** must contain all evaluation points; the code refuses points outside it. A tighter range gives a smaller ρ and therefore a faster evaluation.

Table 1: Measured maximum relative error of the complete transform as a function of the window width w (worst case of $L = 13$ and $L = 185$, 1500 points including both poles, radial rank at full precision). The code picks the smallest w whose $\sigma = 2$ error is below the requested ε. For comparison, the direct double-precision sum has error 1.6×10^{-14} at $L = 185$.

w	$\sigma = 2$ (default)	$\sigma = 3$
4	7×10^{-4}	1×10^{-4}
5	2×10^{-5}	3×10^{-5}
6	4×10^{-6}	7×10^{-7}
7	6×10^{-7}	1×10^{-7}
8	3×10^{-8}	8×10^{-9}
9	4×10^{-9}	6×10^{-10}
10	6×10^{-10}	4×10^{-11}
11	3×10^{-11}	3×10^{-12}
12	3×10^{-12}	3×10^{-13}
13	5×10^{-13}	1×10^{-14}
14	4×10^{-14}	–
15	2×10^{-14}	–

11 Cost and memory

Set-up.

- Legendre functions on $2L + 2$ sample points for all (n, m): about L^3 operations.
- FFTs in θ: about $L^3 \log L$.
- Matrix products of Step 2: about $3\rho L^3$.
- 3ρ two-dimensional FFTs of size $n_{f,1} \times n_{f,2} \approx (2\sigma L)^2$: about $\rho L^2 \log L$.

In practice this takes 0.01 s for $L = 13$ and about 1 s for $L = 185$ (table 2).

Memory. The stored grid has 3ρ channels of about $\frac{1}{2} n_{f,1} \times n_{f,2} \approx 2\sigma^2 L^2$ real numbers each, i.e. about $3\rho \cdot 8L^2 \cdot 8$ bytes for $\sigma = 2$. For $L = 185$ that is 34 MB at $\varepsilon = 10^{-6}$ and 84 MB at machine precision.

Evaluation, per point.

- $2w$ window-weight evaluations;
- $3\rho\, w^2$ multiply–adds for the window sum (the dominant term);
- about ρK operations for the radial values.

For $\varepsilon = 10^{-10}$ ($w = 11$, $\rho \approx 9$) the window sum is about 3 300 multiply–adds. **None of these numbers depends on L.**

Comparison with the direct sum. The direct sum needs about $L^2/2$ terms per point, each costing some 20 operations (recursions and accumulation): about 340 000 operations for $L = 185$. The fast method therefore wins by a large factor at high degree. At low degree the two costs become similar. At $\varepsilon = 10^{-6}$ the fast method already wins at $L = 13$ (1.4×); at $\varepsilon = 10^{-10}$ it wins from $L \approx 20$, and at machine precision from $L \approx 20$–30 (table 4).

Table 2: Set-up (precomputation) time in seconds and size of the stored grid in MB.

	$\varepsilon = 10^{-6}$		$\varepsilon = 10^{-10}$		$\varepsilon = 10^{-13}$	
L	set-up	grid	set-up	grid	set-up	grid
13	0.01	0.2	0.01	0.3	0.01	0.4
20	0.01	0.4	0.01	0.7	0.03	0.9
30	0.02	0.8	0.02	1.6	0.03	2.1
45	0.03	2.3	0.05	3.4	0.06	4.5
70	0.09	5.2	0.13	8.6	0.11	11
100	0.22	10	0.32	19	0.28	23
133	0.45	18	0.53	33	0.49	44
185	0.68	34	0.95	63	0.98	84

12 Accuracy

Error sources. There are three:

- the window (the alias terms of (20)), controlled by w;
- the truncation of the radial SVD, controlled by ρ;
- floating-point rounding, which is small (section 12).

Steps 2 and 3 are otherwise exact.

How we measured it. To measure errors of order 10^{-14} we need a reference that is more accurate than double precision. We evaluated the model with the same recursions in 80-bit "long double" arithmetic, about 19 significant digits, at 1500 random points in the 250–800 km shell, including points exactly at both poles. Errors are reported relative to the largest field value, $\max|\Delta B| / \max|B|$.

Results. Figure 7 and table 1 show the error as a function of w. It falls by about a factor of 10 per extra window point, and is nearly identical for $L = 13$ and $L = 185$. At $w = 14$ it reaches $3\text{–}4 \times 10^{-14}$.

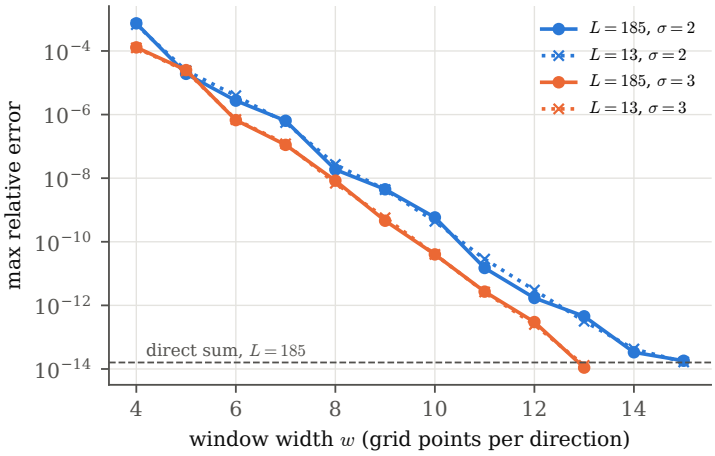


Figure 7: Maximum relative error of the complete transform against the long-double reference, as a function of the window width w. The dashed line is the error of the direct double-precision sum for $L = 185$.

Machine precision. The direct sum computed in double precision is not exact either: rounding in its recursions gives relative errors of 1.1×10^{-15} at $L = 13$, growing to 1.6×10^{-14} at $L = 185$ (9×10^{-10} nT). With $w = 14$ the fast method has error 3.3×10^{-14} at $L = 185$, essentially the same. **At this setting the fast method is as accurate as the direct sum, and still** $65\times$ **faster** (table 4).

13 Performance results

Set-up of the tests.

- **Models.** The IGRF-2025 main field (degrees up to 13, from the IGRF-14 coefficient file). For higher degrees, random lithospheric coefficients are added with a CHAOS-like flat spectrum of $0.5\,\mathrm{nT}^2$ per degree at the surface. Speed does not depend on the coefficient values.
- **Points.** Uniformly random on the sphere, at altitudes between 250 and 800 km.
- **Hardware.** Two CPU cores; both methods use both.
- **Timing.** Times are the best of three runs and exclude the set-up unless stated. The direct sum for large M is timed on a subset and scaled, which is exact because its cost is proportional to M.

Findings. All results are for the default fused loop unless stated; table 3 compares it with the FINUFFT option. (Table 3 comes from a separate run; repeated runs of the same configuration agree to within a few per cent.)

- **FINUFFT option:** the fused loop is 1.1–2.1$\times$ faster than FINUFFT doing the same window sums, typically 1.4$\times$; with FINUFFT the speedups at $L = 185$ are 134$\times$, 65$\times$ and 43$\times$. Both reach the accuracy of the direct sum. At $\varepsilon = 10^{-14}$ FINUFFT's tuned window gives 4.4×10^{-15} at $L = 185$, below the direct sum's own rounding error.
- **Cost per point is independent of** L (fig. 8, right): about 0.3–0.5 μs per point at $\varepsilon = 10^{-6}$, 0.7–0.9 μs at 10^{-10} and 1.1–1.4 μs at 10^{-13}. The direct sum grows from 0.5 μs at $L = 13$ to 86 μs at $L = 185$.
- **Speedups at** $L = 185$ with 10^6 points: 201$\times$ ($\varepsilon = 10^{-6}$), 98$\times$ (10^{-10}) and 65$\times$ (machine precision).
- **IGRF (**$L = 13$**):** the fast method is 1.4$\times$ faster at $\varepsilon = 10^{-6}$, but slower than the direct sum at higher accuracy. For IGRF alone a good direct code remains a sensible choice.
- **Including the set-up** (fig. 9), the method breaks even at about 10^4 points at high degree. At $L = 185$ and 10^5 points it is 8–12$\times$ faster in total, and 37–78$\times$ faster at 10^6 points.
- **The two simplifications** of sections 5.4 and 7 made evaluation 1.4–1.9$\times$ faster and halved the memory, at identical settings (table 5).

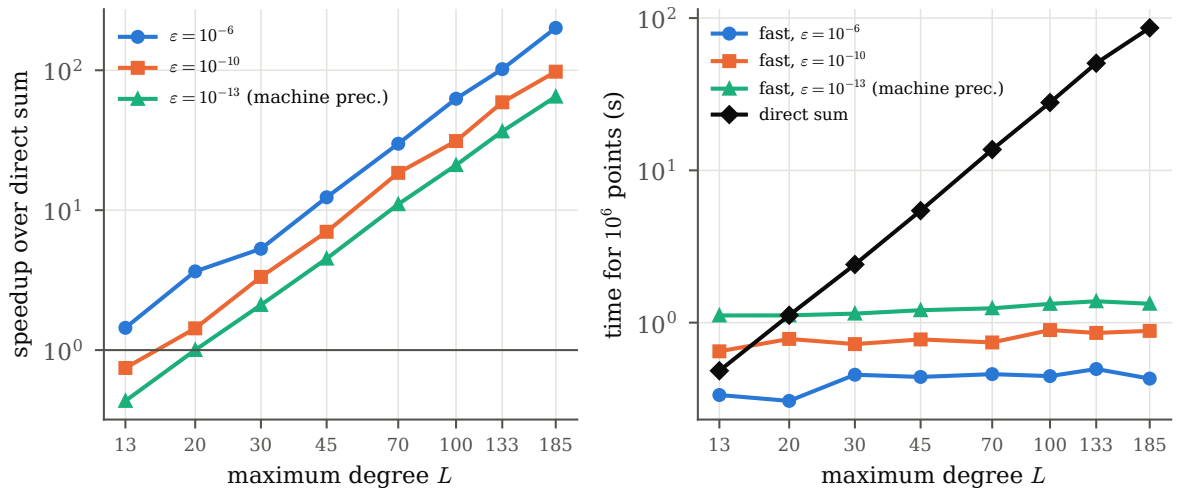


Figure 8: *Left:* speedup over the direct sum for 10^6 points. *Right:* time for 10^6 points. The fast method's cost is flat in L, while the direct sum grows like L^2. With the optional FINUFFT backend, times are about 1.4× longer (table 3).

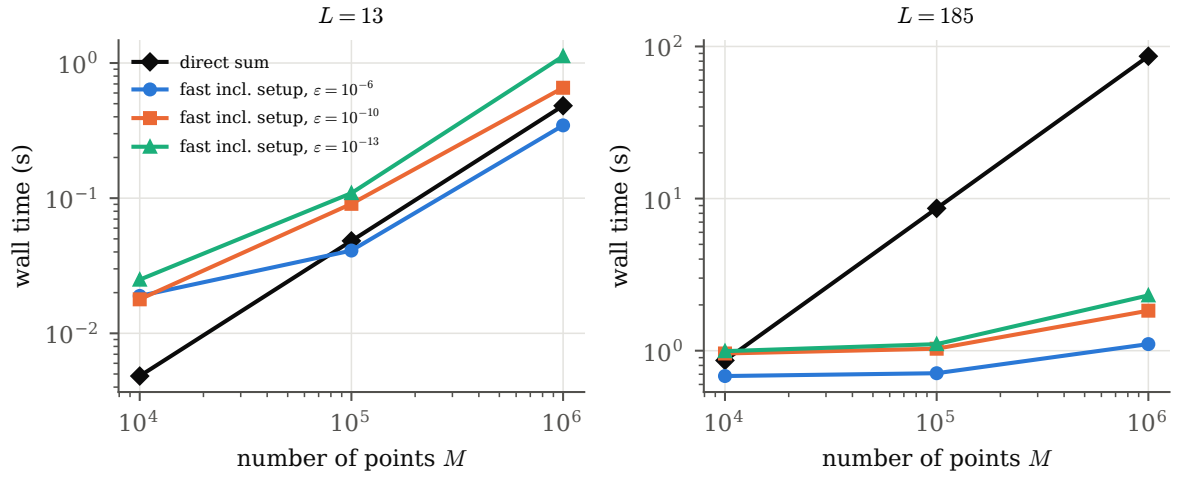


Figure 9: Total time *including the set-up* versus the number of points, for IGRF size ($L = 13$) and a full lithospheric model ($L = 185$).

Table 3: The two implementations of Step 5 at the same requested accuracy ε: FINUFFT's interpolation-only mode (the default) and our own fused loop. $M = 10^6$ points, 250–800 km shell, two cores. Time in seconds; error is the maximum relative error against the long-double reference; speedup is relative to the direct sum.

			FINUFFT (optional)			own fused loop (default)		
L	ε	w	time	error	speedup	time	error	speedup
13	10^{-6}	7	0.47	9.2×10^{-7}	**1.0**	0.33	5.6×10^{-7}	1.5
13	10^{-10}	11	0.97	6.8×10^{-11}	**0.5**	0.68	2.8×10^{-11}	0.7
13	10^{-13}	14	1.56	4.2×10^{-14}	**0.3**	1.22	4.2×10^{-14}	0.4
20	10^{-6}	7	0.74	7.8×10^{-7}	**1.5**	0.36	6.4×10^{-7}	3.1
20	10^{-10}	11	1.27	7.4×10^{-11}	**0.9**	0.74	2.3×10^{-11}	1.5
20	10^{-13}	14	1.46	5.7×10^{-14}	**0.8**	0.92	3.6×10^{-14}	1.2
45	10^{-6}	7	0.61	6.2×10^{-7}	**8.9**	0.42	5.9×10^{-7}	13
45	10^{-10}	11	1.05	6.7×10^{-11}	**5.2**	0.99	1.3×10^{-11}	5.5
45	10^{-13}	14	2.53	6.1×10^{-14}	**2.1**	1.60	3.1×10^{-14}	3.4
100	10^{-6}	7	0.67	8.1×10^{-7}	**41**	0.49	6.0×10^{-7}	57
100	10^{-10}	11	1.28	7.4×10^{-11}	**22**	0.87	1.4×10^{-11}	32
100	10^{-13}	14	1.96	6.8×10^{-14}	**14**	1.39	3.1×10^{-14}	20
185	10^{-6}	7	0.65	1.1×10^{-6}	**134**	0.43	6.3×10^{-7}	200
185	10^{-10}	11	1.33	8.6×10^{-11}	**65**	0.90	1.6×10^{-11}	95
185	10^{-13}	14	2.00	5.9×10^{-14}	**43**	1.42	3.3×10^{-14}	61

Table 4: Main results (default fused loop): $M = 10^6$ points in the 250–800 km shell, two CPU cores. Direct sum: time (s) and relative error against the long-double reference. Fast method, for each requested accuracy ε: window width w, radial rank ρ, evaluation time (s), speedup over the direct sum, and measured maximum relative error against the long-double reference.

	direct sum		fast method					
L	time	error	ε	w	ρ	time	speedup	error
			10^{-6}	7	4	0.33	**1.4**	5.6×10^{-7}
13	0.48	1.1×10^{-15}	10^{-10}	11	6	0.65	**0.7**	2.8×10^{-11}
			10^{-13}	14	8	1.11	**0.4**	4.2×10^{-14}
			10^{-6}	7	4	0.31	**3.7**	6.4×10^{-7}
20	1.12	1.9×10^{-15}	10^{-10}	11	7	0.78	**1.4**	2.3×10^{-11}
			10^{-13}	14	8	1.12	**1.0**	3.6×10^{-14}
			10^{-6}	7	4	0.45	**5.3**	6.5×10^{-7}
30	2.41	2.7×10^{-15}	10^{-10}	11	7	0.72	**3.3**	1.5×10^{-11}
			10^{-13}	14	9	1.15	**2.1**	3.6×10^{-14}
			10^{-6}	7	5	0.44	**12**	5.9×10^{-7}
45	5.44	3.8×10^{-15}	10^{-10}	11	7	0.78	**7.0**	1.3×10^{-11}
			10^{-13}	14	9	1.21	**4.5**	3.1×10^{-14}
			10^{-6}	7	5	0.46	**30**	6.9×10^{-7}
70	13.70	7.1×10^{-15}	10^{-10}	11	8	0.74	**18**	1.5×10^{-11}
			10^{-13}	14	10	1.24	**11**	3.3×10^{-14}
			10^{-6}	7	5	0.45	**63**	6.0×10^{-7}
100	27.89	8.9×10^{-15}	10^{-10}	11	9	0.89	**31**	1.4×10^{-11}
			10^{-13}	14	11	1.33	**21**	3.1×10^{-14}
			10^{-6}	7	5	0.50	**102**	6.1×10^{-7}
133	50.53	1.3×10^{-14}	10^{-10}	11	9	0.86	**59**	1.4×10^{-11}
			10^{-13}	14	12	1.38	**37**	3.1×10^{-14}
			10^{-6}	7	5	0.43	**201**	6.3×10^{-7}
185	86.25	1.6×10^{-14}	10^{-10}	11	9	0.88	**98**	1.6×10^{-11}
			10^{-13}	14	12	1.33	**65**	3.3×10^{-14}

Table 5: Effect of the two simplifications of sections 5.4 and 7: the earlier complex-valued, full-grid implementation ("old") versus the real-valued, half-grid implementation ("new"), at identical window width w, $\sigma = 2$ and radial rank ρ. $M = 10^6$ points; times in seconds, grid sizes in MB (0 means below 0.5 MB).

L	w	ρ	old	new	speedup	old grid	new grid
13	8	5	0.64	0.41	1.56	0	0
13	12	7	1.17	0.73	1.59	1	0
13	14	8	1.69	0.89	1.90	1	0
20	8	5	0.60	0.44	1.36	1	1
20	12	7	1.26	0.79	1.59	1	1
20	14	9	1.76	1.13	1.56	2	1
45	8	6	0.83	0.51	1.63	5	3
45	12	8	1.47	0.88	1.68	7	4
45	14	10	1.92	1.29	1.49	9	5
100	8	7	1.01	0.53	1.91	29	14
100	12	10	1.70	1.13	1.50	41	21
100	14	12	2.12	1.27	1.67	49	25
185	8	7	0.96	0.51	1.87	95	48
185	12	11	1.72	1.03	1.67	149	77
185	14	14	2.69	1.70	1.58	190	98

14 Practical advice and limitations

- **When to use it.** Use the method for models of degree $L \gtrsim 20$ and more than about 10^4 points. For IGRF alone the direct sum is about as fast.
- **Which accuracy.** $\varepsilon = 10^{-6}$ corresponds to about 0.04 nT, well below measurement noise and model uncertainty, and gives the largest speedups. Use $\varepsilon = 10^{-13}$ when results must match a direct computation to the last digits.
- **Radial range.** Set $[r_{\min}, r_{\max}]$ as tight as the data allow. Very thick shells need more radial functions; for those, split the data into thinner shells, each with its own set-up.
- **Time-dependent models.** Coefficients that vary with time, such as IGRF secular variation or the CHAOS core field, can be handled by linearity: evaluate one transform per time basis function and combine the results with per-point weights. For CHAOS, the time-dependent core field has low degree ($n \leq 20$) and can simply be evaluated directly. The static lithospheric part, where nearly all of the cost lies, then uses this method.
- **Not yet implemented.** The current code does not provide the transpose operation needed for least-squares inversion (structure in section 9), the potential V itself, or a GPU version. All are straightforward extensions.

15 Summary

A spherical harmonic model can be evaluated at scattered points much faster than by direct summation, by combining four ideas:

1. **Low-rank radial structure.** The radial factors $(a/r)^{n+2}$ are captured by a few optimal radial functions found by an SVD.
2. **Exact Fourier form.** On the doubled sphere, the angular parts are exact 2-D trigonometric polynomials, regular at the poles.
3. **Pre-sharpening.** Dividing by the window's spectrum and moving to an oversampled grid lets a $w \times w$ window sum reproduce the model, with an error that falls by about a factor of ten per window point.
4. **Economy.** The grid is real-valued and only half of it is stored, and each point is evaluated in one fused pass.

After a set-up of about a second, the cost per point is independent of the degree L. At $L = 185$ the method is 65–200× faster than the direct sum, depending on the accuracy, and it can match the direct sum's own precision.

A Pole-safe recursions for the Legendre functions

Let $x = \cos\theta$ and $s = \sin\theta$; on the doubled sphere s may be negative. Write $Q_n^m = P_n^m / \sin\theta$ for $m \geq 1$.

Diagonal ($n = m$). $P_0^0 = 1$, $\mathrm{d}P_0^0/\mathrm{d}\theta = 0$; $P_1^1 = s$, $\mathrm{d}P_1^1/\mathrm{d}\theta = x$, $Q_1^1 = 1$. For $m \geq 2$, with $f_m = \sqrt{(2m-1)/(2m)}$:

$$P_m^m = f_m\, s\, P_{m-1}^{m-1}, \qquad \frac{\mathrm{d}P_m^m}{\mathrm{d}\theta} = f_m\Big(s\,\frac{\mathrm{d}P_{m-1}^{m-1}}{\mathrm{d}\theta} + x\,P_{m-1}^{m-1}\Big), \qquad Q_m^m = f_m\, s\, Q_{m-1}^{m-1}.$$

Upward in n. With $A_n^m = \dfrac{2n-1}{\sqrt{n^2-m^2}}$ and $B_n^m = \sqrt{\dfrac{(n-1)^2-m^2}{n^2-m^2}}$ (and the B term omitted for $n = m+1$):

$$\begin{aligned} P_n^m &= A_n^m\, x\, P_{n-1}^m - B_n^m\, P_{n-2}^m, \\ \frac{\mathrm{d}P_n^m}{\mathrm{d}\theta} &= A_n^m\Big(x\,\frac{\mathrm{d}P_{n-1}^m}{\mathrm{d}\theta} - s\,P_{n-1}^m\Big) - B_n^m\,\frac{\mathrm{d}P_{n-2}^m}{\mathrm{d}\theta}, \\ Q_n^m &= A_n^m\, x\, Q_{n-1}^m - B_n^m\, Q_{n-2}^m. \end{aligned}$$

The recursion for Q is the same as for P, started from $Q_m^m = P_m^m/s$, which is known in closed form. No step divides by $\sin\theta$.

B Chebyshev points and series for the radial functions

With $t \in [-1,1]$, the Chebyshev polynomials $T_k(t) = \cos(k \arccos t)$ form the natural basis for smooth functions on an interval. Given the values $f_j = f(t_j)$ at the Chebyshev–Lobatto points $t_j = \cos(\pi j/K)$, $j = 0,\dots,K$, the coefficients of the interpolating series $f(t) \approx \sum_{k=0}^{K} a_k T_k(t)$ are

$$a_k = \frac{2}{K}\sum_{j=0}^{K}{}'' f_j \cos\frac{\pi j k}{K},$$

where $\sum''$ halves the first and last terms, and a_0 and a_K are halved as well. For smooth functions the coefficients a_k decrease very quickly [7], which is why a small K suffices.

The SVD of Step 1 is applied to the samples at these points. The weights c_j (equal to $\frac{1}{2}$ at the two end points and 1 otherwise) make the ordinary sum of squares of a column approximate the integral $\int_{-1}^{1} f(t)^2 (1-t^2)^{-1/2}\mathrm{d}t$. The SVD therefore finds radial functions that are optimal in that integral sense, not just at the sample points.

C The three-dimensional method (for information)

Before the 2-D method we developed a version that treats the radius as a third Fourier direction. We include it because it is mathematically neat, and it explains why the 2-D method was preferred.

Representation. Map the radial interval to an angle: $u = u_c + u_d\cos\psi$, $\psi \in [0,\pi]$. Since u^{n+2} is a polynomial of degree $n+2$ in u, it becomes a cosine series of degree $n+2$ in ψ:

$$u^q = (u_c + u_d\cos\psi)^q = \sum_{k=-q}^{q} \rho_k^{(q)} e^{\mathrm{i}k\psi}.$$

Combined with lemma 2, every field component is *exactly* a 3-D trigonometric polynomial,

$$B_c = \mathrm{Re}\sum_{|k|\le K}\sum_{|p|\le L}\sum_{m=0}^{L} F_c[k,p,m]\, e^{\mathrm{i}(k\psi + p\theta + m\varphi)}, \qquad F_c[k,p,m] = \sum_n \rho_k^{(n+2)}\gamma_n^m \widehat{\Theta}_c^{nm}[p],$$

with $K = L+2$ for an exact representation, and much smaller K (4–16) for thin shells.

Algorithm. Build F_c once, then evaluate all points with a single 3-D "non-uniform FFT" (NUFFT) [8, 9, 10]. This is the same pre-sharpening idea as in Step 3, but in three dimensions and with the FFT performed on every call. We used the FINUFFT library.

Why the 2-D method is preferred.

- **Interpolation cost.** The 3-D window has w^3 points per component, against $\rho\, w^2$ channel values in 2-D. On satellite shells $\rho < w$.
- **No per-call FFT.** In 2-D the grid depends only on the model and is computed once. The 3-D library recomputes its FFT on every call, which at $L = 185$ costs as much as interpolating 10^6 points.
- **Better radial basis.** The 3-D method must use a Fourier (Chebyshev) basis in the radial direction; the 2-D method can use the optimal SVD basis.

Table 6 compares the two. The 3-D method remains useful for very thick radial ranges, where ρ would exceed w.

Table 6: The 3-D method (FINUFFT, $\sigma = 2$) versus the fused 2-D method of the main text, $M = 10^6$ points, evaluation times in seconds. K: radial (Chebyshev) degree of the 3-D method. The two were run in separate sessions on the same machine. At the same requested ε the 2-D method is also more accurate (e.g. 1.6×10^{-11} versus $\approx 2 \times 10^{-10}$ at $L = 185$, $\varepsilon = 10^{-10}$).

	$\varepsilon = 10^{-6}$			$\varepsilon = 10^{-10}$		
L	K	3-D	2-D	K	3-D	2-D
13	4	0.63	0.33	6	2.09	0.65
20	4	0.65	0.31	6	1.84	0.78
45	4	0.70	0.44	8	1.80	0.78
100	5	0.82	0.45	10	1.95	0.89
133	6	1.01	0.50	11	2.02	0.86
185	6	1.40	0.43	12	2.97	0.88

D Library transforms versus a fused loop

The 2-D method can also be run with a standard NUFFT library, by passing the 3ρ channels as a batch of separate 2-D transforms. This is correct, but in our tests it was 0.5–0.9$\times$ the speed of the 3-D method, i.e. *slower*. A library treats the transforms of a batch independently. For every channel it recomputes each point's window weights, writes a separate output array, and that array must then be read again for the radial sum. Two changes remove most of this gap while still using the library: moving the FFT into the set-up (interpolation-only mode), and packing two real channels into each complex transform. With both, FINUFFT comes within a factor of about 1.4 of the fused loop (table 3). The remaining gap is structural. Profiling at $L = 185$, $\varepsilon = 10^{-10}$, 10^6 points ($\rho = 9$, i.e. 14 complex transforms) shows:

- 76–82 ms per transform, the same for a batch of 1 or 14, so nothing per point is shared between transforms except the sorting;
- 224 MB of per-channel output that must be read back;
- 0.23 s for the unpacking and radial sum in a separate pass.

The fused loop avoids all three. This is why the purpose-built loop is the default and the library an option.

E Why one approximate step is unavoidable

The FFT is exact and fast because equally spaced points have a special algebraic structure: the factors $e^{2\pi ijk/n}$ can be split and reused. Scattered points have no such structure. Evaluating a sum of harmonics at scattered points is equivalent to evaluating a polynomial at many arbitrary points. Exact fast algorithms for that exist in principle [12], but they lose accuracy exponentially when run in floating-point arithmetic. All practical fast methods for scattered points therefore contain one approximate step whose error can be made arbitrarily small: here, the window sum. The method in this document reduces that step to a short, local sum whose error is set by a single parameter w, and can be pushed down to the rounding error of the direct sum.

F Using the code

The reference implementation is `fastfield.py` (Python with `numpy`, `scipy` and `numba`; `finufft` only for the optional backend):

```
from fastfield import FastField, vector_to_gh
g, h = vector_to_gh(coeff_vector, L)        # or arrays g[n, m], h[n, m] in nT
F = FastField(g, h, rmin=6371.2 + 250, rmax=6371.2 + 800, eps=1e-10)   # set-up, ~1 s at L = 185
# own fused loop (default); backend="finufft" lets FINUFFT do the window sums
Br, Btheta, Bphi = F(r_km, theta_rad, phi_rad)   # theta = colatitude
B_north, B_east, B_down = -Btheta, Bphi, -Br
```

`F.w`, `F.rho` and `F.nbytes` report the window width, the radial rank and the grid size. Optional arguments `backend`, `sigma`, `rank_tol` and (own loop only) `w` override the automatic choices.

References

[1] P. Alken et al., International Geomagnetic Reference Field: the thirteenth generation, *Earth, Planets and Space* 73:49 (2021).

[2] C. C. Finlay, C. Kloss, N. Olsen, M. D. Hammer, L. Tøffner-Clausen, A. Grayver, A. Kuvshinov, The CHAOS-7 geomagnetic field model and observed changes in the South Atlantic Anomaly, *Earth, Planets and Space* 72:156 (2020).

[3] J. W. Cooley, J. W. Tukey, An algorithm for the machine calculation of complex Fourier series, *Math. Comp.* 19:297–301 (1965).

[4] N. Schaeffer, Efficient spherical harmonic transforms aimed at pseudospectral numerical simulations, *Geochem. Geophys. Geosyst.* 14(3):751–758 (2013).

[5] J. R. Driscoll, D. M. Healy, Computing Fourier transforms and convolutions on the 2-sphere, *Adv. Appl. Math.* 15:202–250 (1994).

[6] G. H. Golub, C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press (2013).

[7] L. N. Trefethen, *Approximation Theory and Approximation Practice*, SIAM (2013).

[8] A. Dutt, V. Rokhlin, Fast Fourier transforms for nonequispaced data, *SIAM J. Sci. Comput.* 14(6):1368–1393 (1993).

[9] L. Greengard, J.-Y. Lee, Accelerating the nonuniform fast Fourier transform, *SIAM Review* 46(3):443–454 (2004).

[10] A. H. Barnett, J. Magland, L. af Klinteberg, A parallel nonuniform fast Fourier transform library based on an "exponential of semicircle" kernel, *SIAM J. Sci. Comput.* 41(5):C479–C504 (2019).

[11] S. Kunis, D. Potts, Fast spherical Fourier algorithms, *J. Comput. Appl. Math.* 161:75–98 (2003).

[12] A. Borodin, R. Moenck, Fast modular transforms, *J. Comput. Syst. Sci.* 8:366–386 (1974).